

Script verbale del seminario

Slide 1 - Copertina

“In questo seminario presento un lavoro che ho fatto: scrivere da zero un check personalizzato per clang-tidy. L’idea di fondo è avere un linter che ragiona sul codice e non solo sul testo. Vedremo com’è costruito, come l’ho testato su progetti reali, e, come da suo suggerimento, un confronto con gli LLM su casi concreti.”

Slide 2 - Il problema

“Il punto di partenza è una situazione concreta: in code review capita spesso di trovare `printf` in codice C++ moderno, dove esiste un’alternativa migliore e type-safe, `std::print`. Vorrei che fosse il CI a segnalarlo in automatico, con il fix già pronto. Il problema è che gli strumenti testuali (`grep`, espressioni regolari...) guardano i caratteri, non il significato: appena entrano in gioco scope, template e macro, o danno falsi positivi o si perdono dei casi. Serve qualcosa che ragioni su tipi e contesto, come fa il compilatore.”

Slide 3 - Architettura clang-tidy

“Uno strumento così esiste già, è clang-tidy e qui abbiamo il flusso. Il sorgente entra nel frontend di Clang, che produce un AST arricchito di semantica: tipi risolti, overload selezionati, template istanziati. Sopra l’AST gira il sistema degli AST matcher, un linguaggio dichiarativo per descrivere pattern. Il check vero e proprio è una classe C++ che reagisce ai match e produce diagnostica e fix-it.

I due pezzi in rosso, matcher e check, sono gli unici che ho scritto io; tutto il resto è infrastruttura di Clang. È il vantaggio di avere il compilatore disponibile come libreria.”

Slide 4 - L’AST di Clang

“Vediamo concretamente cos’è questo AST. Con quattro righe si vede l’AST di un `cast (int)3.14`. Sotto al `VarDecl` c’è il `CStyleCastExpr` con il tipo della conversione, e sotto ancora il `FloatingLiteral`. Il punto è che il cast non è una stringa: è un nodo tipato, con la conversione esplicitata.

È questo il livello su cui ragionano i check. Il parsing, il type checking e la name resolution sono già stati fatti, io mi aggancio qui.”

Slide 5 - AST Matchers

“Per descrivere i nodi che mi interessano uso un piccolo DSL. Si legge dall'esterno verso l'interno, come una frase. Parto da `callExpr`, cioè una chiamata di funzione. Dentro c'è `callee`, che è la funzione chiamata. Questa funzione deve essere un `functionDecl`, una dichiarazione di funzione. E infine `hasName :: printf` dice che deve avere proprio il nome `::printf` nel namespace globale. Si compone come un albero ed è dichiarativo: descrivo cosa cercare, non come visitare l'albero.

La cosa importante è che `hasName` lavora sul nome qualificato, quello risolto dopo la name resolution. Quindi un metodo come `Logger::printf` ha un nome diverso e non matcha. È esattamente quello che una regex non può fare.”

Slide 6 - Anatomia di un check

“Ogni check è una classe che eredita da `ClangTidyCheck` e implementa due funzioni: `registerMatchers`, dove dichiaro i pattern, e `check`, che viene chiamata a ogni match. È piccola di proposito: tutta la complessità la gestisce il driver di clang-tidy, io scrivo solo la regola.”

Slide 7 - misc-no-printf

“Questo è il check che ho scritto. Il check si chiama `misc-no-printf`. Segnala le chiamate a `::printf` di `libc`, non segnala `mylib::printf` perché è una funzione diversa in un altro scope, e produce un fix-it che sostituisce il nome con `std::print` di C++23.

L'ho scelto come esempio perché è minimale ma non banale: tocca scope, macro e template, cioè i tre casi che fanno fallire un'analisi testuale.”

Slide 8 - registerMatchers

“Queste sono le poche righe del matcher completo. La parte importante è `hasName :: printf`, che aggancia il nome globale e qualificato, quindi niente collisioni con i namespace utente. L'`unless(isExpansionInSystemHeader())` esclude i match che vengono dagli header di sistema, altrimenti ogni `#include <cstdio>` darebbe diagnostica fuori dal nostro codice. Il `.bind` etichetta il nodo, per ritrovarlo dopo nella funzione `check`.”

Slide 9 - check + diagnostic + fix-it

“E questa è la funzione check, che produce diagnostica e fix-it. Recupero il `CallExpr` che ho etichettato, prendo la posizione del callee (il token `printf`) ed emetto una diagnostica con un `FixItHint::CreateReplacement` che sostituisce solo quell'intervallo con `std::print`.

Sostituisco solo il nome, non l'intera chiamata: così il fix è chirurgico, idempotente, e clang-tidy lo può applicare in automatico su molti file a ogni PR.”

Slide 10 - Divider “Demo”

“A questo punto lo vediamo girare, prima su esempi mirati e poi su progetti reali. Passiamo alla parte pratica.”

Slide 11 - Demo 1: esempi mirati + confronto LLM

“Ho preparato sei snippet di esempio. Lanciando clang-tidy: sul caso con la collisione di namespace ottengo un solo hit, `mylib::printf` viene ignorato. Sul caso più complesso, quello con `using logging::printf` e overload resolution, il check capisce che `::printf("...", 1)` con argomento `int` risolve in realtà al template `logging::printf`, non a `libc`, quindi segnala solo l'unica chiamata davvero a `libc`.

Gli stessi sei snippet li ho dati a due LLM, Claude Opus 4.7 e ChatGPT 5.5, per un totale di 24 run: sei snippet, due modelli, due chat fresche ciascuno, anche per vedere se sono coerenti con se stessi. I risultati sono in tabella. clang-tidy è deterministico al 100%; gli LLM no. Il fix con `std::print`: Claude lo propone come primario in 4 casi su 6, ChatGPT mai. Sullo snippet 6 lo stesso modello, in due chat identiche, dà interpretazioni opposte dell'overload resolution.

C'è anche il rovescio della medaglia: su un altro snippet entrambi gli LLM trovano un bug che il mio check non vede, un `static_cast<int>(2.5)` che tronca a 2, una perdita silenziosa di precisione. È un bug semantico fuori dal mandato del check. Quindi non fanno la stessa cosa, fanno cose diverse.”

Slide 12 - Demo 2: progetti veri

“Stessa cosa su progetti reali, non più esempi giocattolo. Due casi. Il primo è `bear_demo`, un piccolo progetto con build Make: Bear intercetta la build e produce il `compile_commands.json`, poi clang-tidy gira e dà cinque hit su cinque, con zero falsi positivi. È il caso realistico di un progetto con un suo sistema di build a cui si attacca un linter senza riscrivere niente.

Il secondo è `tinyxml2`, una libreria vera presa da GitHub, con build CMake: due unità di compilazione, 28 hit in due decimi di secondo. Il dato che mi interessa è questo: `grep printf` su quel codice trova 42 match, `clang-tidy` ne segnala 28. I 14 di differenza sono commenti, stringhe, `vfprintf`, `snprintf`, macro. Ovvero esattamente i falsi positivi che un linter testuale produrrebbe. È il valore misurabile dell'analisi sull'AST.”

Slide 13 - Test, doc, CI

“Infine, come si integra in un progetto vero. Quattro aspetti. I test sono in stile lit, lo standard di LLVM, e girano dentro la build farm ufficiale. La documentazione è in `.rst`, come tutti i check ufficiali. Per il CI, `--warnings-as-errors='*'` fa fallire la pipeline e blocca il merge se il check trova qualcosa. E il check si compila come parte di `clang-tools-extra`, quindi potrebbe anche essere proposto upstream.”

Slide 14 - Take-away

“In conclusione, tre punti. Quando conviene scrivere un check custom: quando c'è un pattern ricorrente, un vincolo deterministico e un fix riapplicabile. Quando non conviene: se la regola è sfumata, dove è meglio una review umana o un LLM, oppure se è uno stile generico, dove probabilmente esiste già un check ufficiale.

Sul confronto con gli LLM, la conclusione è che non sono sostituti ma complementari: `clang-tidy` è preciso e deterministico dove conta (scope, tipi, overload resolution) mentre gli LLM trovano bug fuori dal mandato del check, come il troncamento `2.5 → 2`, ma sulla precisione di tipo compilatore fluttuano da un'esecuzione all'altra. Un LLM può aiutare a scrivere un nuovo check; `clang-tidy` poi lo applica in modo ripetibile su tutto il codice.

In sostanza, la domanda non è ‘meglio `clang-tidy` o un LLM’, ma che tipo di domanda si sta facendo al codice: se ha una risposta deterministica vince `clang-tidy`, se richiede comprensione del contesto vince l'LLM. Grazie, resto a disposizione per le domande.”

Q&A - domande probabili e risposte preparate

Q: “Quanto ci hai messo a scriverlo?”

“Il check in sé è meno di 50 righe. Il grosso del tempo è andato sul build di LLVM da sorgente (~30 minuti la prima volta), sul lit test integrato nella build farm, e sulle run sui progetti reali. In totale 25-30 ore, ma un secondo check lo farei in 1-2 ore.”

Q: “Su grossi codebase è scalabile?”

“Sì, ed è uno dei punti forti. clang-tidy è parallelizzabile per file, idempotente, e ha un costo per file dell'ordine dei millisecondi una volta che esiste `compile_commands.json`. Con un LLM invece è impraticabile: rate limit, costi token, non determinismo. Su 10^5 file c'è una sola scelta sensata.”

Q: “Perché hai scelto `std::print` come fix e non `std::cout`?”

“Perché `std::print` (C++23) è il rimpiazzo type-safe più diretto di `printf`: mantiene l'idea della format string ma con controllo a compile time. `std::cout` è il fallback C++20, ma cambia stile (operatore `<<`). Volevo un fix il più possibile drop-in.”

Q: “Hai contato anche i falsi negativi del check?”

“Su `tinyxml2` zero, perché `printf` di `libc` è proprio quello che il check cerca. In un caso più sofisticato - chiamata via puntatore a funzione - il check li ignora deliberatamente, perché non sono `callee(functionDecl)` matchabili. È il trade-off precisione contro recall: ho preferito massimizzare la precisione per stare a zero falsi positivi.”

Q: “Il confronto LLM è statisticamente significativo? Solo 24 run.”

“La critica è giusta: 24 run è un campione piccolo. Il pattern ‘ChatGPT non propone mai `std::print` come fix primario’ è 0 su 6 snippet, quindi è un risultato qualitativo, non statistico. Per renderlo significativo servirebbero centinaia di run su decine di snippet. Quello che mostro è un'indicazione qualitativa, robusta su due aspetti: i due LLM danno risposte diverse tra loro, e lo stesso LLM è incoerente con sé stesso tra chat fresche.”

Q: “L'avresti potuto fare con `grep -P` e basta?”

“Sul caso `tinyxml2`: `grep printf` trova 42 match, di cui 14 sono falsi positivi evidenti - commenti, stringhe, `fprintf`, `snprintf`, macro. Sono circa il 33% di rumore. Su un progetto da mille file è ingestibile in code review. Per questo serve l'AST.”

Q: “E con un modello più recente? I test sono su Opus 4.7.”

“Ho rifatto i test con Opus 4.8. È più capace: trova più bug reali fuori dal mandato del check, per esempio che prendere l'indirizzo `&std::printf` è undefined behavior, o che il wrapper template disabilita il controllo `-Wformat` del compilatore. Ed è anche più coerente tra chat fresche. Ma sul caso più difficile, quello con l'overload resolution, continua a non azzeccare che `::printf("...", 1)` risolve al template e non a `libc`, mentre clang-tidy lo fa in modo deterministico. Quindi la non-determinazione specifica si è attenuata, ma la conclusione di fondo regge anzi si rafforza: clang-tidy è preciso sul suo mandato, l'LLM è bravo fuori, e sono complementari.”

Spiegazione dei comandi della demo (per domande dal vivo)

Se il prof chiede cosa fa un comando delle slide 11/12:

- `clang-tidy -checks='-* , misc-no-printf' file.cpp` - disattiva tutti i check (-*) e abilita solo il mio (`misc-no-printf`), così l'output è pulito e mostra solo i miei hit.
 - `bear -- make` - **Bear** (“Build EAR”) intercetta la build lanciata da `make` e registra ogni comando di compilazione, producendo `compile_commands.json`. Serve perché `clang-tidy` deve compilare i file con gli stessi flag del progetto.
 - `compile_commands.json` - il “compilation database”: per ogni file elenca il comando esatto di compilazione (flag, include path, standard). È ciò che permette a `clang-tidy` di vedere il codice come lo vede il compilatore.
 - `clang-tidy -p . file.cpp` - il `-p .` dice a `clang-tidy` dove trovare il `compile_commands.json` (qui nella dir corrente).
 - `cmake -B build -DCMAKE_EXPORT_COMPILE_COMMANDS=ON` - genera la build in `build/` ed esporta direttamente il `compile_commands.json`, senza bisogno di `Bear` (CMake lo sa fare nativamente).
 - `--extra-arg=-isysroot $(xcrun --show-sdk-path)` - su macOS passa a Clang il path dell'SDK di sistema, altrimenti non trova gli header standard (`<stdio>` ecc.) e fallisce. `xcrun --show-sdk-path` stampa quel path.
 - `--warnings-as-errors='*'` - trasforma ogni warning in errore: in CI fa fallire la pipeline e blocca il merge.
-

Note pratiche per la presentazione

1. **Slide 11:** è densa, non leggere la tabella riga per riga. Indica i dati col puntatore: i 24 run, lo 0 su 6 di ChatGPT, il 50/50 dello snippet 6.
 2. **Demo viva:** se c'è tempo, batti i comandi a schermo (slide 11 e 12). Se sei in ritardo, vai al video di backup di 2'30”.
 3. **Tempo:** se vai lungo, la slide 4 (AST dump) è la più sacrificabile. Se sei veloce, espandi la slide 12 sul confronto `grep` vs AST.
 4. **Q&A:** se non sai una risposta, dillo: “non ho misurato esattamente quel caso, proverei prima X e Y”. Onestà meglio di finta sicurezza.
-

Cose da NON dire

- Non dire “questo è un argomento difficile” o “magari non ha capito”.
- Non scusarti per la durata: hai 15 minuti.
- Non dire “ovviamente”: ciò che è ovvio per te può non esserlo per chi ascolta.
- Non leggere le slide parola per parola: servono a chi vuole rivederle dopo, tu parli con chi ascolta.